4.1.5 Λογικοί τελεστές

Περιλαμβάνονται οι λογικοί τελεστές and και or, ο τριαδικός τελεστής και ο τελεστής instanceof. Ο τελεστής instanceof σχετίζεται με το αντικειμενοστρεφές μοντέλο και η παρουσίασή του γίνεται στην ενότητα 13.2.1.

4.1.5.1 Λογικό and

Το λογικό and συμβολίζεται ως "&&", είναι δυαδικός τελεστής και συντάσσεται αποκλειστικά με τελεστέους τύπου boolean. Η τιμή επιστροφής του υπολογίζεται όπως δείχνει ο πίνακας 4.2.

Τελεστέος 1	**Τελεστέος 2**	**Αποτέλεσμα**
TRUE	TRUE	TRUE
TRUE	FALSE	FALSE
FALSE	TRUE	FALSE
FALSE	FALSE	FALSE

Πίνακας 4.2 *Πίνακας αληθείας λογικού and*

Η υλοποίηση του λογικού and ακολουθεί τη λογική βραχέως κυκλώματος (short circuit). Η αξιολόγηση γίνεται από αριστερά προς τα δεξιά. Αν ο πρώτος τελεστέος αρκεί για τον υπολογισμό του αποτελέσματος, ο δεύτερος τελεστέος δεν αξιολογείται. Σύμφωνα με τον πίνακα 2, όταν ο τελεστέος 1, έχει την τιμή false, τότε το αποτέλεσμα της έκφρασης είναι false ανεξάρτητα από την τιμή του δεύτερου τελεστέου. Αντίθετα, όταν ο πρώτος τελεστέος έχει την τιμή true, τότε το αποτέλεσμα της έκφρασης εξαρτάται από την τιμή του δεύτερου τελεστέου.

```
1    static void shortCircuitAnd() { //Κώδικας 4.10
2        int i = 1, j = 2;
3        System.out.println("i++>1 && j--<3 is " + (i++ > 1 && j-- < 3));
4        System.out.println("i= " + i + " j= " + j);
5        i = 1;
6        j = 2;
7        System.out.println("++i>1 && j--<3 is " + (++i > 1 && j-- < 3));
8        System.out.println("i= " + i + " j= " + j);
9    }
```

Κώδικας 4.10*: Short circuit and*

Η γραμμή 3 του κώδικα 4.10 τυπώνει i++>1 && j--<3 is false. Ο πρώτος τελεστέος είναι η λογική έκφραση i++>1, η τιμή του είναι false. Επομένως, ο δεύτερος, δηλαδή το j--<3 δεν θα αξιολογηθεί και άρα δεν θα μειωθεί η τιμή του j με αποτέλεσμα η έξοδος της γραμμής 4 είναι i= 2 j= 2. Αντίθετα, στη γραμμή 7, ο πρώτος τελεστέος του λογικού and είναι ++i>1 και η τιμή του είναι true. Επομένως θα αξιολογηθεί και ο δεύτερος τελεστέος με αποτέλεσμα να μειωθεί το j. Καθώς και ο δεύτερος τελεστέος είναι true, το αποτέλεσμα του λογικού and είναι επίσης true. Επομένως, η γραμμή 7 θα τυπώσει ++i>1 && j--<3 is true και η γραμμή 8 θα τυπώσει i= 2 j= 1.

4.1.5.2 Λογικό or

Το λογικό or συμβολίζεται ως "||", είναι δυαδικός τελεστής και συντάσσεται αποκλειστικά με τελεστέους τύπου boolean. Η τιμή επιστροφής του υπολογίζεται όπως δείχνει ο πίνακας 4.3.

Τελεστέος 1	**Τελεστέος 2**	**Αποτέλεσμα**
TRUE	TRUE	TRUE
TRUE	FALSE	TRUE

FALSE	TRUE	TRUE
FALSE	FALSE	FALSE

Πίνακας 4.3 *Πίνακας αληθείας λογικού or*

Η υλοποίηση του λογικού or ακολουθεί επίσης τη λογική βραχέως κυκλώματος (short circuit). Η αξιολόγηση γίνεται από αριστερά προς τα δεξιά. Αν ο πρώτος τελεστέος αρκεί για τον υπολογισμό του αποτελέσματος, ο δεύτερος τελεστέος δεν αξιολογείται. Σύμφωνα με τον πίνακα 4.3, όταν ο τελεστέος 1, έχει την τιμή true, τότε το αποτέλεσμα της έκφρασης είναι true ανεξάρτητα από την τιμή του δεύτερου τελεστέου. Αντίθετα, όταν ο πρώτος τελεστέος έχει την τιμή false, τότε το αποτέλεσμα της έκφρασης εξαρτάται από την τιμή του δεύτερου τελεστέου.

```
1      static void shortCircuitOr() { //Κώδικας 4.11
2         int i = 1, j = 2;
3         System.out.println("i++>=1 || j--<3 is " + (i++ >= 1 || j-- < 3));
4         System.out.println("i= " + i + " j= " + j);
5         i = 1;
6         j = 2;
7         System.out.println("i++>=1 || j--<3 is " + (i++ > 1 || j-- < 3));
8        System.out.println("i= " + i + " j= " + j);
9      }
```

Κώδικας 4.11*: Short circuit or*

Η γραμμή 3 του κώδικα 4.11, τυπώνει i++>=1 || j--<3 is true. Ο πρώτος τελεστέος είναι η λογική έκφραση i++>=1, η τιμή του είναι true. Επομένως, ο δεύτερος, δηλαδή το j--<3 δεν θα αξιολογηθεί και άρα δεν θα μειωθεί η τιμή του j με αποτέλεσμα η έξοδος της γραμμής 4 να είναι i= 2 j= 2. Αντίθετα, στη γραμμή 7, ο πρώτος τελεστέος του λογικού or είναι i++>1 και η τιμή του είναι false. Επομένως θα αξιολογηθεί και ο δεύτερος τελεστέος με αποτέλεσμα να μειωθεί το j. Καθώς ο δεύτερος τελεστέος είναι true, το αποτέλεσμα του λογικού or είναι επίσης true. Επομένως, η γραμμή 7 θα τυπώσει i++>1 && j--<3 is true και η γραμμή 8 θα τυπώσει i= 2 j= 1.

4.1.5.3 Ο τριαδικός τελεστής

Ο τριαδικός τελεστής (ternary operator) συμβολίζεται ως “?:”. Όπως φανερώνει το όνομά του, συντάσσεται με τρεις τελεστέους. Ο πρώτος είναι μια έκφραση τύπου boolean, π.χ. i<j, ο δεύτερος η τιμή που επιστρέφει ο τελεστής αν ο πρώτος τελεστέος έχει τιμή true και ο τρίτος τελεστέος δηλώνει την τιμή επιστροφής αν ο πρώτος τελεστέος έχει την τιμή false. Στον κώδικα 4.12 δίνουμε ένα παράδειγμα χρήσης του τριαδικού τελεστή. Στη γραμμή 3, ο πρώτος τελεστέος είναι η λογική έκφραση i<j. Στο συγκεκριμένο παράδειγμα, η έκφραση αυτή έχει τιμή true, επομένως επιστρέφεται η τιμή μετά το ?, δηλαδή η τιμή του i. Αν όμως η τιμή i<j ήταν false, τότε θα επιστρεφόταν ο τρίτος τελεστέος, δηλαδή το j, η τιμή μετά τον χαρακτήρα ‘:’.

```
1      static void ternaryOp() { //Κώδικας 4.12
2          int i = 10, j = 2;
3          int minIJ = i < j ? i : j;
4          System.out.println(minIJ);
5          double d = 5.2;
6          double max = i > d ? i : d;
7          System.out.println(max);
8      }
```

Κώδικας 4.12*: Ο τριαδικός τελεστής*

Ο τριαδικός τελεστής μπορεί να επιστρέψει δεδομένα οποιουδήποτε τύπου με τον περιορισμό πως οι τύποι του δεύτερου και τρίτου τελεστέου είναι συμβατοί μεταξύ τους. Αν οι τύποι αυτοί είναι ίδιοι μεταξύ τους, τότε ο τύπος επιστροφής του τελεστή είναι ίδιος με τον τύπο του δεύτερου και τρίτου τελεστέου. Αν δεν

είναι ίδιοι, τότε ο τύπος επιστροφής είναι ο τύπος με τη μεγαλύτερη χωρητικότητα. Για παράδειγμα, στη γραμμή 6 του κώδικα 4.12, ο τελεστής επιστρέφει το i, τύπου int, εφόσον i>d και το d, τύπου double, εφόσον !(i>d). Ανεξάρτητα από ποια τιμή θα επιστραφεί, ο τύπος της θα είναι double.

4.1.6 Τελεστές ψηφίου

Υποστηρίζονται τέσσερις τελεστές ψηφίου, ο τελεστής ψηφίου AND, ο τελεστής ψηφίου OR, ο τελεστής ψηφίου XOR (Exclusive OR) και ο τελεστής συμπληρώματος. Οι τρεις πρώτοι είναι δυαδικοί, ενώ ο τελεστής συμπληρώματος είναι μοναδιαίος. Οι τελεστές ψηφίου δέχονται ως τελεστέους όλους τους ακέραιους τύπους και τον τύπο boolean. Οι ακέραιοι τύποι μπορούν να συνδυαστούν μεταξύ τους με τους τελεστές ψηφίου, όχι όμως και με τους boolean. Οι τελεστές ψηφίου όταν συντάσσονται με boolean λειτουργούν ως λογικοί τελεστές πλήρους αξιολόγησης. Οι τελεστές ψηφίου αντιμετωπίζουν τους τελεστέους τους σαν ψηφιοχάρτες (bitmaps) και κάνουν υπολογισμούς ψηφίο προς ψηφίο. Όταν συντάσσονται με ακέραιους τύπους, αν έστω και ένας από τους τελεστέους είναι long, τότε επιστρέφουν long αλλιώς int. Όταν συντάσσονται με boolean, επιστρέφουν boolean.

4.1.6.1 Τελεστής ψηφίου AND

Ο τελεστής ψηφίου AND (bitwise AND) συμβολίζεται με τον χαρακτήρα '&'. Ο τελεστής ακολουθεί τον πίνακα αληθείας του AND, δηλαδή αν και τα δύο ψηφία έχουν την τιμή 1, τότε το αντίστοιχο ψηφίο του αποτελέσματος έχει την τιμή 1. Αν έστω και ένα από τα ψηφία έχει την τιμή 0, τότε το αντίστοιχο ψηφίο του αποτελέσματος έχει την τιμή 0. Η γραμμή 4 του κώδικα 4.13 τυπώνει 0. Ο ψηφιοχάρτης του i αποτελείται μόνο από μηδενικά ψηφία. Τα μηδενικά αυτά ψηφία συνδυάζονται με τα αντίστοιχα ψηφία του j. Επομένως, όλα τα ψηφία του αποτελέσματος έχουν την τιμή 0.

```
1   static void bitwiseAnd() { //Κώδικας 4.13
2       int i = 0, j = 1232;
3       int k = i & j;
4       System.out.println("i&j=" + k);
5       int i1 = 0b00000000000000000000000000001010;
6       int j1 = 0b00000000000000011000011100001111;
7       System.out.println("i1=" + i1 + " " + "j1=" + j1);
8       System.out.println("i1&j1=" + (i1 & j1));
9       char d1 = 'A';
10      int m = i & d1;
11      System.out.println("i&d=" + m);
12      long b1 = 0;
13      long rslt = b1 & i;
14      System.out.println(rslt);
15  }
```

***Κώδικας 4.13**: Ο τελεστής ψηφίου AND*

Ένας άλλος τρόπος για να καταλάβουμε καλύτερα τους τελεστές ψηφίου είναι να ορίσουμε ακέραιες μεταβλητές σε δυαδική μορφή όπως έχουμε κάνει στις γραμμές 159 και 160. Στη γραμμή 7, τυπώνουμε τα i1 και j1 και βλέπουμε πως i1=10 και j1=100111; Το αποτέλεσμα του τελεστή μεταξύ των δύο αυτών μεταβλητών είναι ένας ψηφιοχάρτης που έχει 1 μόνο στις θέσεις που έχει 1 και η i1 και η j1. Όλες οι υπόλοιπες θέσεις καταλαμβάνονται από το 0. Οι θέσεις στις οποίες έχουν 1 και η i1 και η j1, ταυτίζονται με τις θέσεις που έχει 1 η i1. Επομένως, i1&j1==i1. Πράγματι, η γραμμή 8 τυπώνει i1&j1=10.

Όπως έχουμε ήδη αναφέρει, όλοι οι ακέραιοι τύποι είναι δεκτοί ως τελεστέοι. Στη γραμμή 10 εφαρμόζεται ο τελεστής μεταξύ μιας μεταβλητής τύπου int και μίας τύπου char. Το αποτέλεσμα είναι τύπου int και η τιμή του 0. Στη γραμμή 13 εφαρμόζεται ο τελεστής μεταξύ μιας μεταβλητής τύπου int και μιας τύπου long. Το αποτέλεσμα είναι τύπου long.

4.1.6.2 Τελεστής ψηφίου OR

Ο τελεστής ψηφίου OR (bitwise OR) συμβολίζεται με τον χαρακτήρα '|'. Ισχύουν και εδώ κατ' αναλογία, όσα παρατίθενται για τον τελεστή ψηφίου AND. Η διαφορά έγκειται στο ότι εδώ ακολουθείται ο πίνακας αληθείας του OR, δηλαδή αν ένα τουλάχιστον ψηφίο έχει την τιμή 1, τότε το αποτέλεσμα είναι 1, διαφορετικά είναι 0.

4.1.6.3 Τελεστής ψηφίου XOR:

Ο τελεστής ψηφίου XOR (bitwise XOR) συμβολίζεται με τον χαρακτήρα '^'. Ισχύουν και εδώ κατ' αναλογία όσα παρατίθενται για τον τελεστή ψηφίου AND. Η διαφορά έγκειται στο ότι εδώ ακολουθείται ο πίνακας αληθείας του XOR που παρατίθεται αμέσως από κάτω στον πίνακα 4.4, δηλαδή αν ένα μόνο ψηφίο έχει την τιμή 1, τότε το αποτέλεσμα είναι 1, διαφορετικά είναι 0.

bit 1	**bit 2**	**bit1^bit2**
1	1	0
1	0	1
0	1	1
0	0	0

Πίνακας 4.4 *Πίνακας XOR*

4.1.6.4 Τελεστής ψηφίου NOT

Ο τελεστής ψηφίου NOT (bitwise NOT) ή τελεστής συμπληρώματος (complement operator) είναι μοναδιαίος, συμβολίζεται με τον χαρακτήρα '~' και αντιστρέφει όλα τα bits του τελεστέου του. Ο τελεστής δεν συντάσσεται με μεταβλητές τύπου boolean.

4.1.7 Τελεστές διολίσθησης

Υποστηρίζονται οι τελεστές, διολίσθηση δεξιά, διολίσθηση αριστερά και μη προσημασμένη διολίσθηση δεξιά.

4.1.7.1 Διολίσθηση δεξιά

Ο τελεστής δεξιάς διολίσθησης (right shift) είναι δυαδικός, συμβολίζεται ως ">>"και συντάσσεται με ακέραιους τύπους. Εφόσον ο πρώτος τελεστέος του είναι long επιστρέφει long, διαφορετικά επιστρέφει int. Η εφαρμογή του τελεστή ενεργεί ως εξής: Καταρχάς δεν επηρεάζει τους τελεστέους αλλά δημιουργεί ένα αντίγραφο του ψηφιοχάρτη του πρώτου τελεστέου. Στη συνέχεια προκαλεί διολίσθηση προς τα δεξιά στο αντίγραφο του πρώτου τελεστέου κατά τον αριθμό των bits που καθορίζει η τιμή του δεύτερου τελεστέου. Έτσι, με τη διολίσθηση χάνεται ένας αριθμός από bits από τη δεξιά πλευρά του αντιγράφου. Τα bits που χάνονται αντικαθίστανται στην αριστερή πλευρά του αντιγράφου με τιμή την τιμή του πρώτου bit του, δηλαδή την τιμή του προσήμου. Ας σημειωθεί εδώ πως η Java χρησιμοποιεί για την κωδικοποίηση ακέραιων την παράσταση μέτρο και πρόσημο για τις θετικές τιμές και την παράσταση συμπληρώματος ως προς δύο (two's complement) για τις αρνητικές. Επομένως, η τιμή του προσήμου είναι 0 για τους θετικούς ακέραιους και 1 για τους αρνητικούς. Για λεπτομέρειες σχετικά με την κωδικοποίηση των ακέραιων σε μορφή συμπληρώματος ως προς δύο, ανατρέξτε στο παράρτημα 1.

```
  1   static void rightShift() { //Κώδικας 4.14
  2       int i = 3;
  3       int j = -3;
  4       System.out.println("Integer.toBinaryString(" + i + ")=" +
Integer.toBinaryString(i));
  5       System.out.println("Integer.toBinaryString(" + i + ">>1)=" +
Integer.toBinaryString(i >> 1));
  6       System.out.println("Integer.toBinaryString(" + j + ")=" +
Integer.toBinaryString(j));
```

```
  7       System.out.println("Integer.toBinaryString(" + j + ">>1)=" +
Integer.toBinaryString(j >> 1));
  8   }
```

***Κώδικας 4.14**: Ο τελεστής δεξιάς διολίσθησης*

Στον κώδικα 4.14 έχουμε ορίσει τις ακέραιες μεταβλητές i και j. Στη γραμμή 4 εκτυπώνεται ο ψηφιοχάρτης της μεταβλητής i με τη βοήθεια της συνάρτησης toBinaryString. Η έξοδος αυτής της γραμμής είναι Integer.toBinaryString(3)=11. Πράγματι, η αναπαράσταση του 3 με τη μορφή συμπληρώματος του δύο είναι 11. Βέβαια, δεδομένου, ότι ο int στην Java έχει μέγεθος 32 bit, η πραγματική αναπαράσταση στη μνήμη είναι 00000000000000000000000000000011. Απλώς η Java δεν τυπώνει τυχόν μηδενικά στην αρχή ενός αριθμού (leading zeroes) δεδομένου ότι αυτά δεν επηρεάζουν την τιμή του. Η γραμμή 5 εκτυπώνει Integer.toBinaryString(3>>1)=1. Στη μεταβλητή i έγινε διολίσθηση δεξιά κατά 1 bit, επομένως από το αποτέλεσμα χάθηκε το τελευταίο 1, ενώ παράλληλα προστέθηκε στην αρχή ένα μηδέν. Ο πλήρης ψηφιοχάρτης του αποτελέσματος είναι 00000000000000000000000000000001.

Στη γραμμή 6 τυπώνεται ο ψηφιοχάρτης του -3. Πιο συγκεκριμένα, η έξοδος της γραμμής 6 είναι Integer.toBinaryString(-3)=11111111111111111111111111111101. Μετά τη διολίσθηση δεξιά κατά 1, το τελευταίο 1 χάνεται και προστίθεται στην αρχή του αποτελέσματος το ψηφίο 1. Έτσι, η έξοδος της γραμμής 7 είναι Integer.toBinaryString(-3>>1)=11111111111111111111111111111110. Ο ψηφιοχάρτης αυτός αντιστοιχεί στη δεκαδική τιμή -2.

4.1.7.2 Διολίσθηση αριστερά

Ο τελεστής αριστερής διολίσθησης (left shift) είναι δυαδικός, συμβολίζεται ως “<<”και συντάσσεται με ακέραιους τύπους. Η διαφορά του από τη διολίσθηση δεξιά είναι πως τα bits χάνονται από αριστερά και προστίθενται δεξιά. Επιπλέον, τα bits που προστίθενται έχουν την τιμή μηδέν ανεξάρτητα από το πρόσημο του πρώτου τελεστέου. Κατά τα λοιπά, ισχύουν κατ’ αναλογία όσα ισχύουν και για τον τελεστή δεξιάς διολίσθησης.

```
    static void leftShift() { //Κώδικας 4.15
        int i = 3;
        int j = -3;
        System.out.println("Integer.toBinaryString(" + i + ")=       " +
Integer.toBinaryString(i));
        System.out.println("Integer.toBinaryString(" + i + "<<1)=" +
Integer.toBinaryString(i << 1));
        System.out.println("Integer.toBinaryString(" + j + ")=" +
Integer.toBinaryString(j));
        System.out.println("Integer.toBinaryString(" + j + "<<1)=" +
Integer.toBinaryString(j << 1));
        System.out.println("value of j<<1=" + (j << 1));
    }
```

***Κώδικας 4.15**: Τελεστής αριστερής διολίσθησης*

Η έξοδος του κώδικα 4.15 είναι η ακόλουθη:

```
Integer.toBinaryString(3)=11
Integer.toBinaryString(3<<1)= 110
Integer.toBinaryString(-3)=11111111111111111111111111111101
Integer.toBinaryString(-3<<1)= 11111111111111111111111111111010
value of j<<1=-6
```

Προσέξτε τον ψηφιοχάρτη του j, δηλαδή του -3. Μετά τη διολίσθηση έχει χαθεί ένα ψηφίο με τιμή 1 ενώ στο τέλος του ψηφιοχάρτη έχει προστεθεί ένα ψηφίο με τιμή 0. Η τιμή του j μετά τη διολίσθηση έχει γίνει -6.

4.1.7.3 Μη προσημασμένη διολίσθηση δεξιά

Ο τελεστής μη προσημασμένης δεξιάς διολίσθησης (unsigned right shift) συμβολίζεται με >>> και διαφέρει από τον τελεστή δεξιάς διολίσθησης κατά το ότι τα ψηφία που αντικαθιστούν όσα χάνονται δεν εξαρτώνται από το πρόσημο του πρώτου τελεστέου αλλά έχουν πάντα την τιμή 0.

Ας σημειωθεί πως στην Java δεν υπάρχει τελεστής μη προσημασμένης αριστερής διολίσθησης, καθώς αυτός θα ήταν πανομοιότυπος με την αριστερή διολίσθηση.

4.1.8 Σύνθετη εκχώρηση

Υποστηρίζεται μια σειρά δυαδικών τελεστών σύνθετης εκχώρησης (compound assignment), δηλαδή τελεστών που συνδυάζουν την εκχώρηση με μία άλλη πράξη. Στον πίνακα 4.5 παρουσιάζονται οι τελεστές σύνθετης εκχώρησης μαζί με την ισοδύναμη έκφραση που αντιστοιχεί στον κάθε ένα.

Τελεστής	**Παράδειγμα Χρήσης**	**Ισοδύναμη έκφραση**
+=	i+=5	i=i+5
-=	i-=k	i=i-k
*=	i*=k	i=i*k
/=	i/=k	i=i/k
%=	i%=k	i=i%k
&=	i&=k	i=i&k
|=	i|=k	i=i|k
<<=	i<<=k	i=i<<k
>>=	i>>=k	i=i>>k
>>>=	i>>>=k	i=i>>>k

Πίνακας 4.5 *Τελεστές Σύνθετης Εκχώρησης*

4.2. Βασική Διαμόρφωση εξόδου

Συχνά χρειάζεται να διαμορφώσουμε τα δεδομένα πριν τα εμφανίσουμε. Για παράδειγμα, σε μια εφαρμογή μπορεί να επιθυμούμε να εμφανίσουμε τους πραγματικούς αριθμούς με δύο δεκαδικά ψηφία. Σε αυτήν την ενότητα παρουσιάζονται δύο εναλλακτικοί τρόποι για τη διαμόρφωση της εξόδου.

4.2.1 Η printf

Η printf αποτελεί τον παραδοσιακό τρόπο διαμόρφωσης της εξόδου με την έννοια ότι υποστηρίζεται και στην C [2]. Ας ξεκινήσουμε την παρουσίαση με ένα παράδειγμα:

```
1   static void stringF() { ////Κώδικας 4.16
2       System.out.printf("%s%n", "Hello");
3       System.out.printf("%-10s%10S%n", "Hello", "Hello");
4       System.out.printf("%10.3S%n", "Hello");
5   }
```

Κώδικας 4.16*: Διαμόρφωση εξόδου του τύπου String*

Η πρώτη παράμετρος σε όλες τις κλήσεις της printf στον κώδικα 4.16 ονομάζεται προσδιοριστής διαμόρφωσης (format specifier). Ο προσδιοριστής διαμόρφωσης περιλαμβάνει μια σειρά από κανόνες διαμόρφωσης (formatting rules). Κάθε κανόνας σηματοδοτεί την αρχή του με τον χαρακτήρα %. Στη γραμμή 2, ο προσδιοριστής διαμόρφωσης έχει την τιμή %s%n. Επομένως περιλαμβάνει δύο κανόνες: Ο πρώτος κανόνας εκφράζεται με τον χαρακτήρα s και μας λέει πως η διαμόρφωση αφορά αλφαριθμητική

σειρά. Ο χαρακτήρας s σε αυτό το πλαίσιο ονομάζεται χαρακτήρας μετατροπής (conversion character). Άλλοι χαρακτήρες μετατροπής είναι ο d για διαμόρφωση ακέραιων, ο f για διαμόρφωση πραγματικών αριθμών, ο c για διαμόρφωση χαρακτήρων και ο b για διαμόρφωση τιμών boolean. Ωστόσο, οι χαρακτήρες μετατροπής c και b χρησιμοποιούνται σπάνια. Ο δεύτερος κανόνας περιλαμβάνει τον χαρακτήρα n που προκαλεί αλλαγή γραμμής.

Η δεύτερη παράμετρος στη γραμμή 2 είναι το String που επιθυμούμε να τυπώσουμε. Καθότι ο κανόνας διαμόρφωσης στην πρώτη γραμμή δεν καθορίζει τίποτα περισσότερο παρά μόνο ότι θα εμφανίσουμε μια αλφαριθμητική σειρά και θα προβούμε σε αλλαγή γραμμής, εκείνο που θα δούμε ως έξοδο είναι απλώς η σειρά Hello.

Στη γραμμή 3, μετά τον προσδιοριστή διαμόρφωσης έχουμε μια σειρά από String. Είναι γενικό χαρακτηριστικό της printf ότι μετά τον προσδιοριστή διαμόρφωσης μπορεί να δέχεται πολλές παραμέτρους. Για περισσότερες πληροφορίες επάνω σε αυτό, δείτε την ενότητα 7.3.3. Κάθε μία από αυτές τις παραμέτρους προορίζεται για να εμφανιστεί. Τη διαμόρφωσή τους ρυθμίζει ο προσδιοριστής διαμόρφωσης, δηλαδή η πρώτη παράμετρος. Στη γραμμή 3 λοιπόν έχουμε αυτόν τον προσδιοριστή διαμόρφωσης %-10s%10S%n. Περιλαμβάνει τρεις κανόνες. Ο τρίτος, όπως εξηγήθηκε παραπάνω, προκαλεί αλλαγή γραμμής της εξόδου. Ο πρώτος αφορά τη δεύτερη παράμετρο της printf και ο δεύτερος την τρίτη. Οι δύο πρώτοι κανόνες περιλαμβάνουν το 10. Αυτό προσδιορίζει το πλάτος στο οποίο θα εμφανιστεί η αντίστοιχη παράμετρος. Τα δεδομένα στοιχίζονται σε αυτό το πλάτος στη δεξιά πλευρά, δηλαδή το Hello σε πλάτος 10 θα εμφανιστεί ως ␣␣␣␣␣Hello, που ␣ συμβολίζει ένα διάστημα (space). Αν ωστόσο επιθυμούμε τα δεδομένα μας να στοιχηθούν στα αριστερά του πλάτους, τότε τοποθετούμε τον χαρακτήρα – (μείον) όπως έχουμε κάνει στον πρώτο κανόνα. Σημειώστε πως σε αυτό το πλαίσιο ο χαρακτήρας μείον ονομάζεται σηματοφόρος (flag). Έτσι το πρώτο Hello θα εμφανισθεί ως Hello ␣␣␣␣␣. Αν επιθυμούμε η σειρά να εμφανιστεί με κεφαλαία, τότε ως χαρακτήρα διαμόρφωσης δίνουμε το S αντί του s, όπως κάνουμε στον δεύτερο κανόνα. Στη γραμμή 4 το πλάτος ακολουθείται από την ακρίβεια (precision). Η ακρίβεια δηλώνεται με μια τελεία ακολουθούμενη από έναν ακέραιο.
Η έξοδος του κώδικα 4.16 έχει ως εξής:

```
Hello
Hello          HELLO
       HEL
```

Το γενικό συντακτικό ενός κανόνα διαμόρφωσης έχει ως εξής:

```
%[σηματοφορείς][πλάτος][.ακρίβεια]χαρακτήρας μετατροπής
```

Από αυτά τα στοιχεία, μόνο ο χαρακτήρας % που δηλώνει την αρχή του κανόνα και ο χαρακτήρας μετατροπής που δηλώνει τον τύπο των δεδομένων ή αλλαγή γραμμής είναι υποχρεωτικά. Οι σηματοφορείς, το πλάτος και η ακρίβεια είναι προαιρετικά στοιχεία. Ωστόσο, η σειρά με την οποία τα στοιχεία τοποθετούνται μέσα σε έναν κανόνα είναι υποχρεωτικά και αυτή δείχνει το γενικό συντακτικό του κανόνα. Ας δούμε και μερικά άλλα χρήσιμα παραδείγματα:

```
1   static void intF() { //Κώδικας 4.17
2       System.out.printf("ακέραιος: %d%n", 1000000L);
3       System.out.printf("ακέραιος: %,d%n", 1000000L);
4       System.out.printf(Locale.ENGLISH,"ακέραιος: %,d%n", 1000000L);
5   }
```

***Κώδικας 4.17**: Διαμόρφωση εξόδου ακέραιων τύπων*

Όπως φαίνεται στον κώδικα 4.17, ένας προσδιοριστής διαμόρφωσης μπορεί να περιλαμβάνει και απλές αλφαριθμητικές σειρές οι οποίες απλώς αποτελούν μέρος της εξόδου. Στη γραμμή 2 δεν δίνουμε κάποια ιδιαίτερη διαμόρφωση, οπότε η έξοδος αυτής της γραμμής είναι απλά 1000000. Στη γραμμή 3 όμως, έχουμε

τοποθετήσει τον σηματοφόρο (κόμμα). Με αυτόν τον σηματοφόρο ζητάμε ο αριθμός να εμφανιστεί με διαχωριστικά χιλιάδων, οπότε η έξοδος εδώ θα είναι 1.000.000. Επειδή όμως τα διαχωριστικά χιλιάδων δεν είναι ίδια για όλες τις χώρες, μας δίνεται η δυνατότητα να χρησιμοποιήσουμε μια άλλη έκδοση της printf που μας επιτρέπει να καθορίσουμε ποιο ακριβώς διαχωριστικό θα χρησιμοποιήσουμε. Έτσι, η γραμμή 4 εμφανίζει 1,000,000.

```
1     static void realF() { //Κώδικας 4.18
2       System.out.printf("%f%n", 5.1473);
3       System.out.printf("%(010.2f%n", -5.1473);
4       System.out.printf("%-10.2f%n", 5.1473);
5   }
```

***Κώδικας 4.18**: Διαμόρφωση εξόδου πραγματικών τύπων*

Η έξοδος του κώδικα 4.18 έχει ως εξής:

```
5,147300
(00005,15)
5,15
```

Στη γραμμή 3, ο σηματοφόρος '(' (αριστερή παρένθεση) έχει ως αποτέλεσμα να εμφανίζονται οι αρνητικοί αριθμοί εντός παρενθέσεων. Το δε 0 που ακολουθεί καλύπτει το πλάτος με μηδενικά στα αριστερά. Ο σηματοφόρος 0 δεν συντάσσεται με στοίχιση στα αριστερά. Κατά τα λοιπά, με όσα έχουμε συζητήσει μέχρι εδώ, είναι εύκολο να ερμηνεύσετε την έξοδο του κώδικα 4.18.

4.2.2 Η DecimalFormat

Ένας εναλλακτικός τρόπος για διαμόρφωση δεκαδικών αριθμών για έξοδο βασίζεται στην κλάση DecimalFormat. Η DecimalFormat παρέχει ευκολίες στη διαμόρφωση των αριθμών για έξοδο, εισάγει ωστόσο κάποιον βαθμό πολυπλοκότητας στον κώδικά μας.

```
1     static void outputFormat() { //Κώδικας 4.19
2         DecimalFormat f = new DecimalFormat("###,###.##");
3         double d1 = 9.566, d2 = 9.273;
4         System.out.println(f.format(d1) + "+" + f.format(d2) + " = " +
f.format(d1 + d2));
5
6         d1 = 9.565;
7         d2 = 9.275;
8         System.out.println(f.format(d1) + "+" + f.format(d2) + " = " +
f.format(d1 + d2));
9
10         d1 = 0.275;
11         d2 = 0.385;
12         System.out.println(f.format(d1) + "+" + f.format(d2) + " = " +
f.format(d1 + d2));
13
14         d1 = 9_856_327.563;
15         d2 = 5627869.289;
16         System.out.println(f.format(d1) + "+" + f.format(d2) + " = " +
f.format(d1 + d2));
17
18         f.applyPattern("###,###.###");
19         System.out.println(f.format(d1) + "+" + f.format(d2) + " = " +
f.format(d1 + d2));
20
```